Trabalho - Teste de Integração

Sistema de Agendamento e Controle de Voos de Helicóptero

Aluno: João Victor Bagatim
Disciplina: Qualidade e Teste de Software
Professor: Hugo Menezes Barra

1. Introdução e Contextualização do Case

O trabalho utiliza o case da AeroLog Helicópteros, empresa responsável por voos de táxi aéreo e transporte offshore. Foi implementado um sistema simples em Python para agendar voos, verificar a situação da aeronave e registrar os agendamentos em um banco SQLite.

O objetivo dos testes de integração foi verificar se o serviço, a validação e o banco funcionam corretamente em conjunto. Uma falha nessa comunicação pode permitir horários sobrepostos, aceitar uma aeronave em inspeção ou deixar um registro incorreto no banco, prejudicando o planejamento da operação.

2. Arquitetura da Solução e Integrações

O BookingService recebe a aeronave, o piloto e os horários do voo. Em seguida, chama o ValidationModule, que consulta o Database para verificar o cadastro, a manutenção e a disponibilidade. Se as regras forem atendidas, o serviço solicita a inserção do voo com status AGENDADO.

Módulo	Responsabilidade
database.py	Cria as tabelas aeronaves, pilotos e agendamentos; consulta os dados e insere o agendamento.
validation.py	Verifica o período, a existência da aeronave, o status LIBERADO e a ausência de conflito.
booking_service.py	Coordena a validação e a persistência dentro de uma transação SQLite.

A integração ocorre diretamente entre os módulos Python, sem APIs HTTP e sem simular o banco. O contexto transacional confirma a operação no sucesso e desfaz alterações em caso de exceção. As recusas de negócio ocorrem antes da inserção.

Foram adotados horários locais sem fuso e intervalos que permitem um voo começar quando o anterior termina. O escopo segue as regras obrigatórias; não foram criados limites de peso ou passageiros, pois o enunciado não informa esses valores.

3. Matriz de Cenários de Testes

Dados fictícios da fixture: aeronave 1 (PR-ABC, LIBERADO), aeronave 2 (PR-DEF, EM_INSPECAO) e piloto 1. Todos os horários abaixo pertencem ao dia 29/09/2026.

Cenário	Dados de entrada / ação	Comportamento esperado	Resultado obtido
1. Agendamento válido	Aeronave 1, piloto 1, das 10h às 11h, sem voo prévio.	Retornar o identificador e persistir o voo como AGENDADO.	Aprovado. Registro e dados confirmados por outra conexão SQLite.
2. Conflito de horário	Cadastrar um voo da aeronave 1 das 10h às 11h e repetir o pedido.	Gerar erro de conflito e não inserir o segundo voo.	Aprovado. Exceção esperada; as três tabelas mantiveram o estado anterior.
3. Aeronave em manutenção	Solicitar voo da aeronave 2 das 10h às 11h.	Recusar por manutenção e manter o banco inalterado.	Aprovado. Exceção esperada e nenhuma alteração nas tabelas.
4. Isolamento com fixture	Executar duas vezes o teste que agenda a aeronave 1 das 10h às 11h.	Cada execução deve começar sem agendamentos e terminar com um voo.	Aprovado nas duas execuções, usando bases temporárias distintas.
5. Horários adjacentes	Agendar a aeronave 1 das 10h às 11h e das 11h às 12h.	Aceitar ambos, pois não há sobreposição.	Aprovado. Dois agendamentos registrados.
6. Intervalo inválido	Aeronave 1, início às 11h e fim às 10h.	Recusar o período e preservar o banco.	Aprovado. Exceção esperada e estado preservado.
7. Aeronave inexistente	Aeronave 99, piloto 1, das 10h às 11h.	Recusar o cadastro inexistente e preservar o banco.	Aprovado. Exceção esperada e estado preservado.

Os cenários 1 a 4 atendem aos testes obrigatórios. Os demais verificam condições simples da implementação. A suíte possui oito execuções, pois o cenário de isolamento foi parametrizado para executar duas vezes.

4. Evidências de Execução e Cobertura

Os testes foram executados em Windows com Python 3.13.7, pytest 9.1.1 e pytest-cov 7.1.0. Resultado: oito testes aprovados nas duas execuções.

Execução da suíte: python -m pytest -v

```
$ python -m pytest -v
==================================== test session starts ====================================
collecting ... collected 8 items

tests/test_flight_integration.py::test_agendamento_valido PASSED                           [ 12%]
tests/test_flight_integration.py::test_conflito_de_horario PASSED                          [ 25%]
tests/test_flight_integration.py::test_aeronave_em_manutencao PASSED                       [ 37%]
tests/test_flight_integration.py::test_isolamento_com_fixture[1] PASSED                    [ 50%]
tests/test_flight_integration.py::test_isolamento_com_fixture[2] PASSED                    [ 62%]
tests/test_flight_integration.py::test_intervalos_adjacentes PASSED                        [ 75%]
tests/test_flight_integration.py::test_intervalo_invalido PASSED                           [ 87%]
tests/test_flight_integration.py::test_aeronave_inexistente PASSED                         [100%]

==================================== 8 passed in 0.66s ======================================
```

Cobertura: python -m pytest -v --cov=src --cov-branch --cov-report=term-missing

```
$ python -m pytest -v --cov=src --cov-branch --cov-report=term-missing
Name                     Stmts   Miss Branch BrPart  Cover   Missing
--------------------------------------------------------------------
src\__init__.py              0      0      0      0   100%
src\booking_service.py      11      0      0      0   100%
src\database.py             16      0      0      0   100%
src\validation.py           13      0      8      0   100%
--------------------------------------------------------------------
TOTAL                       40      0      8      0   100%
Coverage JSON written to file evidencias/coverage.json
==================================== 8 passed in 1.18s ======================================
```

A cobertura foi de 100% das 40 instruções e dos oito ramos de decisão dos módulos em src. Isso indica que a suíte percorreu o código implementado, mas não demonstra ausência de todos os defeitos ou segurança para uso em uma operação real.

5. Conclusão e Lições Aprendidas

Os testes confirmaram a integração entre serviço, validação e persistência para os cenários executados. Os agendamentos válidos foram gravados e as tentativas recusadas preservaram os dados existentes.

O principal cuidado foi o isolamento do banco. A fixture utiliza tmp_path para criar uma base exclusiva por teste, prepara os cadastros e, ao final, fecha a conexão e remove o arquivo. Assim, os testes podem repetir os mesmos horários sem herdar registros de outras execuções. Essa atividade mostrou a importância de verificar tanto o retorno do serviço quanto o estado persistido no banco.

Repositório: github.com/joaobagatim/Trabalho-Assincrono-Teste-de-Integracao